

AWS Cloud Misconfiguration Detection Lab

Project Overview

In this lab, I set up a cloud security monitoring environment using AWS free tier services. My goal was to simulate two common real-world misconfigurations, detect them using AWS-native logging and threat detection tools, and then remediate them.

The two misconfigurations I simulated were:

- An S3 bucket made publicly accessible, exposing a file containing sensitive data
- An IAM user granted AdministratorAccess, violating the principle of least privilege

Tools used in this lab:

- AWS S3 - object storage, used to simulate a data exposure misconfiguration
- AWS IAM - identity and access management, used to simulate privilege escalation
- AWS CloudTrail - API activity logging, used to detect both misconfigurations
- AWS GuardDuty - threat detection service, used to review findings

Lab Objectives

1. Create an AWS free tier account and configure a billing alert
2. Enable CloudTrail to log all API activity in my account
3. Enable GuardDuty for automated threat detection
4. Intentionally misconfigure an S3 bucket to expose it publicly
5. Intentionally create an over-privileged IAM user
6. Use CloudTrail Event History to identify both misconfigurations in the logs
7. Review GuardDuty findings dashboard for flagged activity
8. Remediate both misconfigurations and document the fixes
9. Write a findings report summarizing what I detected and how I fixed it

Step-by-Step Lab Walkthrough

Step 1: AWS Account

I went to aws.amazon.com and created a new free tier account. As a precaution, I immediately navigated to the budget dashboard and created a billing alert set to \$50.00 so I would be notified before incurring any unexpected charges. (I also made an alert for a \$10 mark)

Budget amount

Your budgeted amount: **\$50.00**

To change your budgeted amount, go back to step 2.

▼ Alert #1

Remove

Set alert threshold

Threshold

When should this alert be triggered?

100

% of budgeted amount ▼

Trigger

How should this alert be triggered?

Actual ▼

Summary: When your actual cost is greater than **100.00% (\$50.00)** of your **budgeted amount (\$50.00)**, the alert threshold will be exceeded.

Step 2: Enable AWS CloudTrail

I navigated to the CloudTrail service in the AWS Management Console. I created a new trail and configured it to store log files in a newly created S3 bucket. Enabling CloudTrail means that every API call made in my account from this point forward is recorded, including any changes I make to S3 bucket permissions or IAM user policies.

☐	lab1-trail	US East (Ohio)	Yes	arn:aws:cloudtrail:us-east-2:087991250416:trail/lab1-trail	Disabled	No	aws-cloudtrail-logs-087991250416-18b22959	-	-	Logging
-----------------------	----------------------------	----------------	-----	--	----------	----	---	---	---	---------

Step 3: Enable AWS GuardDuty

I navigated to the GuardDuty service in the AWS Console and clicked Enable GuardDuty. GuardDuty is a managed threat detection service that analyzes CloudTrail logs, VPC flow logs, and DNS logs to automatically surface suspicious activity.

 You've successfully enabled GuardDuty. 

Introducing the new AWS Security Hub 



The new Security Hub is your unified cloud security solution featuring prioritized risk detection, central management of Security Hub CSPM, GuardDuty and Inspector, and attack path visualization. It also offers cost optimization through unified pricing across security services.

[Compare pricing](#) 

[Try Security Hub](#) 

Summary [Info](#)

Updated a few seconds ago 

Today 

View and analyze security trends based on GuardDuty findings in your AWS environment.

Overview

 Attack sequences - *new*

0

Resources with findings

0

Total findings

0

Accounts with findings

0

Step 4: Create the S3 Misconfiguration

I navigated to the S3 service and created a new bucket. During setup, I disabled the Block all public access setting, which is on by default. I then created a text file on my local machine called `employee_records.txt` containing fake employee names and fake Social Security Numbers to simulate sensitive data. I uploaded this file to the bucket and configured it to be publicly accessible.

✓ Successfully created bucket "matthew-labbucket-2026" ✕

matthew-labbucket-2026 [Info](#)

[Objects](#) | [Metadata](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (0)

[↻](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open ↗](#) [Delete](#) [Actions ▾](#)

[Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
No objects					
You don't have any objects in this bucket.					
Upload					

✓ Successfully edited public access ✕
View details below.

Make public: status [Close](#)

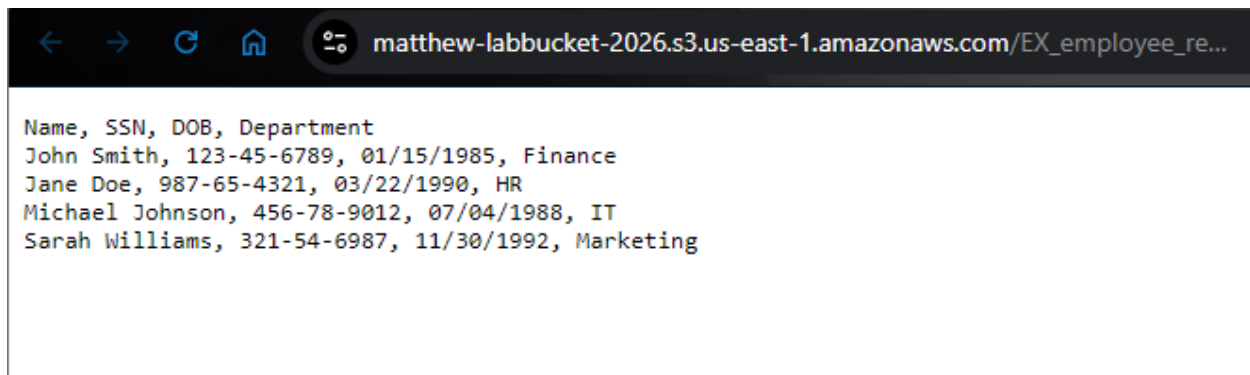
ⓘ After you navigate away from this page, the following information is no longer available.

Summary

Source s3://matthew-labbucket-2026	Successfully edited public access ✓ 1 object, 209.0 B	Failed to edit public access 0 objects
--	---	--

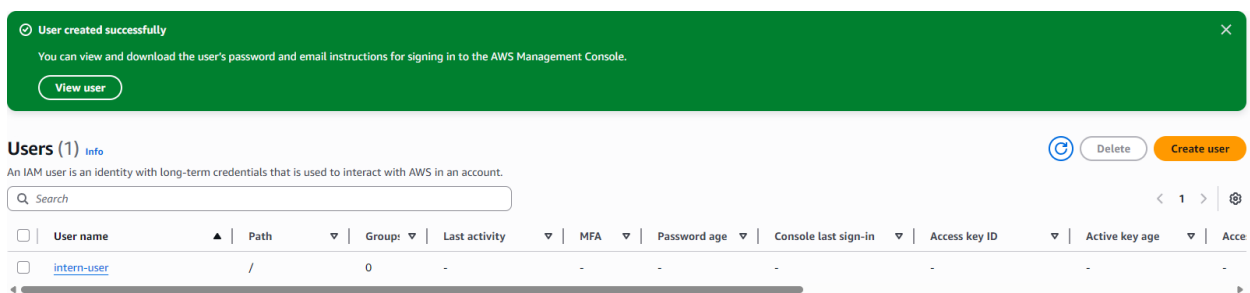
To confirm the misconfiguration worked, I copied the public URL of the file and opened it in my browser. I was able to access the contents of the file without any authentication. This simulates the type of exposure that caused the Capital One breach in 2019, where an improperly configured S3 bucket allowed unauthorized access to the personal data of over 100 million customers.

[link](#)



Step 5: Create the IAM Misconfiguration

I navigated to the IAM service and created a new user named intern-user. I attached the AdministratorAccess managed policy to this user, giving a simulated intern full administrative control over my entire AWS account. This violates the principle of least privilege, which states that users should only be granted the minimum level of access required to perform their job functions.



Step 6: Detect the Misconfigurations Using CloudTrail

I navigated to CloudTrail and opened the Event History view, which shows a real-time log of all API calls made in my account. I searched for the following events:

- PutBucketAcl - logged when I changed the S3 bucket's permissions to allow public access
- CreateUser - logged when I created the intern-user IAM account

Event history (78) [Info](#)

[Download events](#)[Query in Lake](#)[Create Athena table](#)

Event history shows you the last 90 days of management events.

Lookup attributes

Event source

Q s3.amazonaws.com



Filter by date and time

[Clear filter](#)

< 1 2 >

☐	Event name	Event time	User name	Event source
☐	GetBucketOwnershipControls	March 31, 2026, 15:33:01 (UTC-...	root	s3.amazonaws.com
☐	GetBucketVersioning	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	ListTagsForResource	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	CreateBucket	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	PutBucketPublicAccessBlock	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	GetBucketPublicAccessBlock	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	GetBucketObjectLockConfiguration	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	PutBucketEncryption	March 31, 2026, 15:33:00 (UTC-...	root	s3.amazonaws.com
☐	GetAccountPublicAccessBlock	March 31, 2026, 15:31:06 (UTC-...	root	s3.amazonaws.com
☐	ListBuckets	March 31, 2026, 15:31:02 (UTC-...	root	s3.amazonaws.com

```
15     },
16     "eventTime": "2026-03-31T19:33:00Z",
17     "eventSource": "s3.amazonaws.com",
18     "eventName": "PutBucketPublicAccessBlock",
19     "awsRegion": "us-east-1",
20     "sourceIPAddress": "108.56.254.184",
21     "userAgent": "[Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.",
22     "requestParameters": {
23       "publicAccessBlock": "",
24       "bucketName": "matthew-labbucket-2026",
25       "PublicAccessBlockConfiguration": {
26         "xmlns": "http://s3.amazonaws.com/doc/2006-03-01/",
27         "RestrictPublicBuckets": false,
28         "BlockPublicPolicy": false,
29         "BlockPublicAcls": false,
30         "IgnorePublicAcls": false
```

CreateUser [Info](#)

Details [Info](#)

Event time

March 31, 2026, 15:43:16 (UTC-04:00)

User name

root

Event name

CreateUser

Event source

iam.amazonaws.com

AWS access key

ASIARI7FP5XYHOKUJ3B4

Source IP address

108.56.254.184

Event ID

06fb4753-ed7a-4508-a462-6561f9111673

Request ID

1dc8cde5-b52b-4a0e-953f-00266c3897e9

AWS region

us-east-1

Error code

-

Read-only

false

```
},
"eventTime": "2026-03-31T19:43:16Z",
"eventSource": "iam.amazonaws.com",
"eventName": "CreateUser",
"awsRegion": "us-east-1",
"sourceIPAddress": "108.56.254.184",
"userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3",
"requestParameters": {
  "userName": "intern-user"
},
"responseElements": {
  "user": {
    "path": "/",
    "userName": "intern-user",
    "userId": "AIDARI7FP5XYCMP5F4S3Z",
    "arn": "arn:aws:iam::087991250416:user/intern-user",
    "createDate": "Mar 31, 2026, 7:43:16 PM"
  }
}
```

Step 7: Review GuardDuty Findings

I navigated to the GuardDuty service and checked the Findings dashboard. If no findings had fired organically during the lab, I went to Settings and selected Generate Sample Findings to populate the dashboard with realistic example alerts. I reviewed the findings and noted the severity ratings, finding types, and the AWS resources involved.

Findings (2) [Info](#)

Saved filters

Apply saved filters

Filter findings

Status

Current

Threat type

All findings

Create suppression

☐	Title	Severity	Finding type	Resource	Count	Account ID	Last seen
☐	The API ListManagedNotificationEvents was invoked using root credentials.	Low	Policy:IAMUser/RootCredentialUsage	Access Key: ASIARI7FP5XYFBE2L3BI	220	087991250416	8 minutes ago
☐	Amazon S3 Block Public Access was disabled for the S3 bucket matthew-labbucket-2026.	Low	Policy:S3/BucketBlockPublicAccessDisabled	Access Key: ASIARI7FP5XYM3GTPFJO	1	087991250416	28 minutes ago

Step 8: Remediate Both Misconfigurations

After documenting the detections, I remediated both issues:

S3 Fix: I navigated back to the S3 bucket, opened the Permissions tab, and re-enabled the Block all public access setting. I also deleted the publicly accessible employee_records.txt file.

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

Edit Block public access (bucket settings) [Info](#)

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☒ Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☒ Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☒ Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Cancel

Save changes

✔ Successfully deleted objects
View details below.

Delete objects: status

Close

ⓘ After you navigate away from this page, the following information is no longer available.

Summary

Source

s3://matthew-labbucket-2026

Successfully deleted

✔ 1 object, 209.0 B

Failed to delete

0 objects

Failed to delete

Configuration

⊕ Failed to delete (0)

🔍 Find objects by name

Name	Folder	Type	Last modified	Size	Error
------	--------	------	---------------	------	-------

No objects failed to delete.

IAM Fix: I navigated back to the IAM service, opened the intern-user account, detached the AdministratorAccess policy, and replaced it with the ReadOnlyAccess managed policy.

✓ 1 policy added
✕

intern-user [Info](#)
Delete

Summary

ARN <code>arn:aws:iam::087991250416:user/intern-user</code> Access key 1 Create access key Last console sign-in -	Console access Disabled Created March 31, 2026, 15:43 (UTC-04:00)
---	---

<
Permissions
Groups
Tags
Security credentials
Las
>

Permissions policies (1) 🔄 Remove Add permissions ▼

Permissions are defined by policies attached to the user directly or through groups.

🔍 Search

Filter by Type
 All types ▼

<
1
>
⚙️

☐	Policy name 🔗	▲	Type
☐	+ 📦 ReadOnlyAccess		AWS managed - job function

I verified both fixes by attempting to access the S3 file URL again (access denied) and reviewing the intern-user permissions in the IAM console.

← → 🔄 🏠 🌐 matthew-labbucket-2026.s3.us-east-1.amazonaws.com/EX_employee_records.txt.txt...
☆ 📁 🔍 🚫

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```

<?xml version="1.0" encoding="UTF-8" ?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Request has expired</Message>
  <X-Amz-Expires>300</X-Amz-Expires>
  <Expires>2026-03-31T19:46:23Z</Expires>
  <ServerTime>2026-03-31T20:04:52Z</ServerTime>
  <RequestId>9KSH6HHXEDSAEXXB</RequestId>
  <HostId>tvk5Mar+jEM5fpze3HOK+BfGEuKYSQZrNeR87P0MgseGcWRnQ7t7X8u1i7rdJw7QnLNGixjXbZkqpE3Sj3UKvIMAu4vkuVA</HostId>
</Error>

```

Findings Report

Executive Summary

During this lab exercise, I introduced two cloud misconfigurations into an AWS free tier environment and then detected and remediated them using AWS-native tools. Both misconfigurations are representative of real-world vulnerabilities commonly found in cloud environments during security assessments.

Finding 1: Publicly Accessible S3 Bucket

Misconfiguration: I disabled the Block all public access setting on an S3 bucket and uploaded a file containing simulated sensitive employee data. The file was accessible to anyone on the internet without authentication.

Detection: CloudTrail logged a PutBucketAcl event at the time the bucket was made public. The event record included the bucket name, the time of the change, and the user identity that made the change.

Business Risk: An exposed S3 bucket can lead to a large-scale data breach. The Capital One breach in 2019 was caused by a similar misconfiguration and exposed the personal data of over 100 million customers. In a regulated environment, this type of exposure would likely trigger HIPAA, PCI DSS, or state-level breach notification requirements.

Remediation: I re-enabled the Block all public access setting on the bucket and deleted the exposed file. Access was verified as denied upon attempting to reload the public file URL.

Finding 2: Over-Privileged IAM User

Misconfiguration: I created an IAM user named intern-user and attached the AdministratorAccess policy, granting full administrative control over the entire AWS account.

Detection: CloudTrail logged a CreateUser event when the account was created and an AttachUserPolicy event when AdministratorAccess was assigned. Both events were visible in the CloudTrail Event History.

Business Risk: An over-privileged user account creates a significant attack surface. If the intern-user credentials were compromised through phishing or credential stuffing, an attacker would have unrestricted access to all AWS services, data, and infrastructure. This violates the principle of least privilege, a foundational control in NIST CSF, CIS Controls, and ISO 27001.

Remediation: I detached the AdministratorAccess policy and replaced it with ReadOnlyAccess, limiting the user to view-only permissions. The change was reflected immediately in the IAM console.

Recommendations

- Enforce MFA on all IAM users, especially those with elevated permissions
- Enable AWS Config to continuously monitor resource configurations and alert on drift from a secure baseline
- Apply S3 Block Public Access at the account level rather than per-bucket to prevent accidental exposure
- Implement IAM Access Analyzer to automatically flag overly permissive policies
- Review CloudTrail logs regularly using automated alerting to detect configuration changes in real time

Frameworks Referenced

- NIST CSF 2.0 - Protect (PR.AC), Detect (DE.CM)
- CIS Controls v8 - Control 3 (Data Protection), Control 5 (Account Management)
- AWS Well-Architected Framework - Security Pillar

Reflection

This lab gave me hands-on experience with two of the most common causes of cloud security incidents: misconfigured storage buckets and over-privileged user accounts. I was able to see how quickly a misconfiguration can be introduced and how CloudTrail provides an audit trail that makes detection possible after the fact.

The most important takeaway for me was understanding how GuardDuty and CloudTrail work together. CloudTrail captures every API call as a raw log, while GuardDuty applies machine learning and threat intelligence to surface the most important events automatically. In a real SOC environment, analysts use both tools together to triage and investigate cloud security incidents.